

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of
Science and Commerce
Department of Information Technology (B.Sc.I.T Semester IV)

Enterprise Java

Practical – IV

Roll No.: T010	Name: sagar kandari
Class: TYIT	Batch: 01
Date of Assignment: 18/07/26	Date/Time of Submission 18/07/26

Problem Statement :

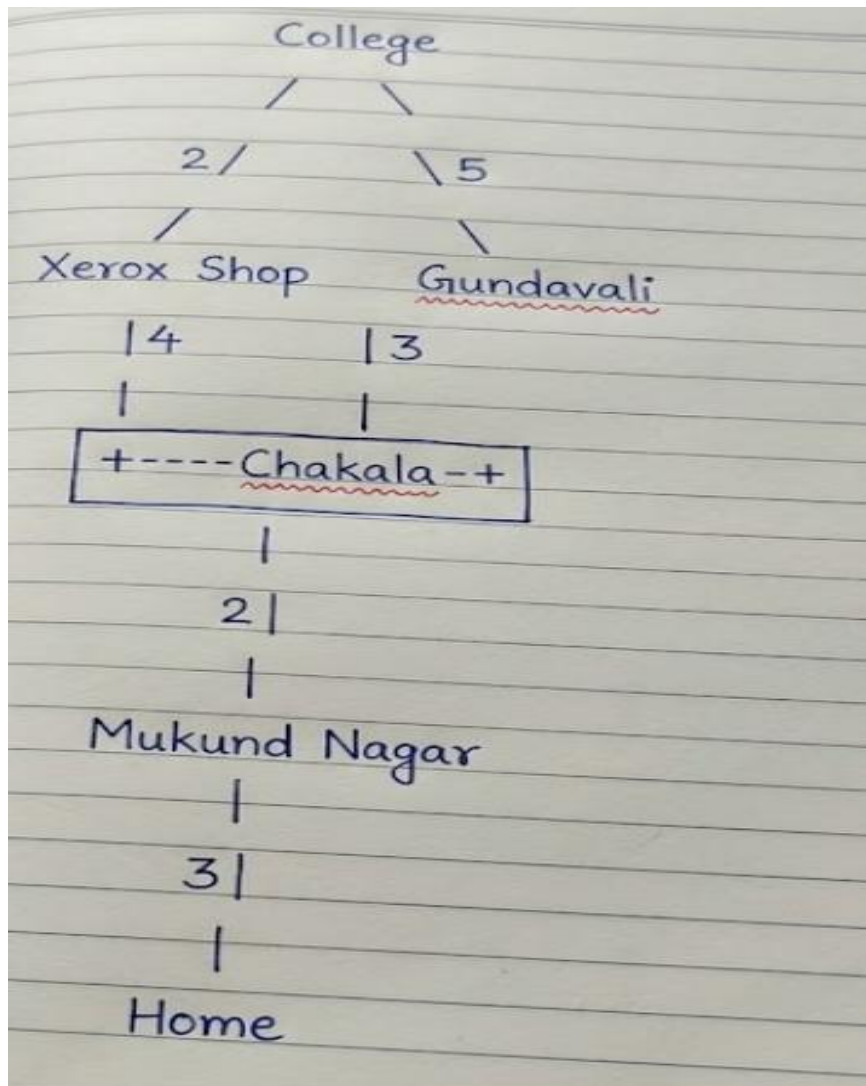
Write Python programs to determine the route from **College** to **Home** using the following search algorithms:

- Breadth-First Search (BFS)
- Depth-First Search (DFS)
- Uniform Cost Search (UCS)
- Greedy Best-First Search
- A* Search

For each algorithm display:

- Path from College to Home
- Total Path Cost
- Number of Nodes Explored

- Graph Diagram :



- Edge Cost Table

From	To	Cost
College	Xerox Shop	2
College	Gundavali	5
Xerox Shop	Chakala	4
Gundavali	Chakala	3
Chakala	Mukund Nagar	2
Mukund Nagar	Home	3

- Heuristic Table (for Greedy and A*)

Node	Heuristic $h(n)$
College	9
Xerox Shop	7
Gundavali	6
Chakala	4
Mukund Nagar	2
Home	0

- Brief Description of Each Algorithm
Breadth First Search (BFS)

Uses a Queue.

Explores nodes level by level.

Finds the shortest path in terms of number of edges.

- **Python Source Code**
- Import Required Libraries:

```
from collections import deque
import heapq
```

- **BFS**

```
# ----- BFS -----
def bfs(start, goal):
    queue = deque([(start, [start], 0)])
    visited = set()
    explored = 0

    while queue:
        node, path, cost = queue.popleft()

        if node in visited:
            continue

        visited.add(node)
        explored += 1

        if node == goal:
            return path, cost, explored

        for neighbor, w in graph[node]:
            if neighbor not in visited:
                queue.append((neighbor, path + [neighbor], cost + w))
```

- DFS

```
def dfs(start, goal):
    stack = [(start, [start], 0)]
    visited = set()
    explored = 0

    while stack:
        node, path, cost = stack.pop()

        if node in visited:
            continue

        visited.add(node)
        explored += 1

        if node == goal:
            return path, cost, explored

        for neighbor, w in reversed(graph[node]):
            if neighbor not in visited:
                stack.append((neighbor, path + [neighbor], cost + w))
```

- UCS

```
# ----- UCS -----
def ucs(start, goal):
    pq = [(0, start, [start])]
    visited = set()
    explored = 0

    while pq:
        cost, node, path = heapq.heappop(pq)

        if node in visited:
            continue

        visited.add(node)
        explored += 1

        if node == goal:
            return path, cost, explored

        for neighbor, w in graph[node]:
            if neighbor not in visited:
                heapq.heappush(pq, (cost + w, neighbor, path + [neighbor]))
```

- Greedy Best-First Search

```
# ----- Greedy Best First -----  
def greedy(start, goal):  
    pq = [(heuristic[start], start, [start], 0)]  
    visited = set()  
    explored = 0  
  
    while pq:  
        h, node, path, cost = heapq.heappop(pq)  
  
        if node in visited:  
            continue  
  
        visited.add(node)  
        explored += 1  
  
        if node == goal:  
            return path, cost, explored  
  
        for neighbor, w in graph[node]:  
            if neighbor not in visited:  
                heapq.heappush(  
                    pq,  
                    (heuristic[neighbor], neighbor, path + [neighbor], cost + w)  
                )
```

- A* Search

```
# ----- A* Search -----  
def astar(start, goal):  
    pq = [(heuristic[start], 0, start, [start])]  
    visited = set()  
    explored = 0  
  
    while pq:  
        f, g, node, path = heapq.heappop(pq)  
  
        if node in visited:  
            continue  
  
        visited.add(node)  
        explored += 1  
  
        if node == goal:  
            return path, g, explored  
  
        for neighbor, w in graph[node]:  
            if neighbor not in visited:  
                new_g = g + w  
                new_f = new_g + heuristic[neighbor]  
                heapq.heappush(  
                    pq,  
                    (new_f, new_g, neighbor, path + [neighbor])  
                )
```

Program Output

Screenshot or console output for each algorithm

```
=====
BFS
=====
Path : College -> Xerox Shop -> Chakala -> Mukund Nagar -> Home
Total Cost : 11
Nodes Explored : 6
Sagar Kandari T010

=====
DFS
=====
Path : College -> Xerox Shop -> Chakala -> Mukund Nagar -> Home
Total Cost : 11
Nodes Explored : 6
Sagar Kandari T010

=====
UCS
=====
Path : College -> Xerox Shop -> Chakala -> Mukund Nagar -> Home
Total Cost : 11
Nodes Explored : 6
Sagar Kandari T010
...
Path : College -> Xerox Shop -> Chakala -> Mukund Nagar -> Home
Total Cost : 11
Nodes Explored : 6
Sagar Kandari T010
```

Writeup Question Answers. (Include in documentation only)

Additional Task:

Google Maps Comparison with A*

Search Google Maps Route

Source: Mvlu College

Destination: Home

Distance: 3.9 km

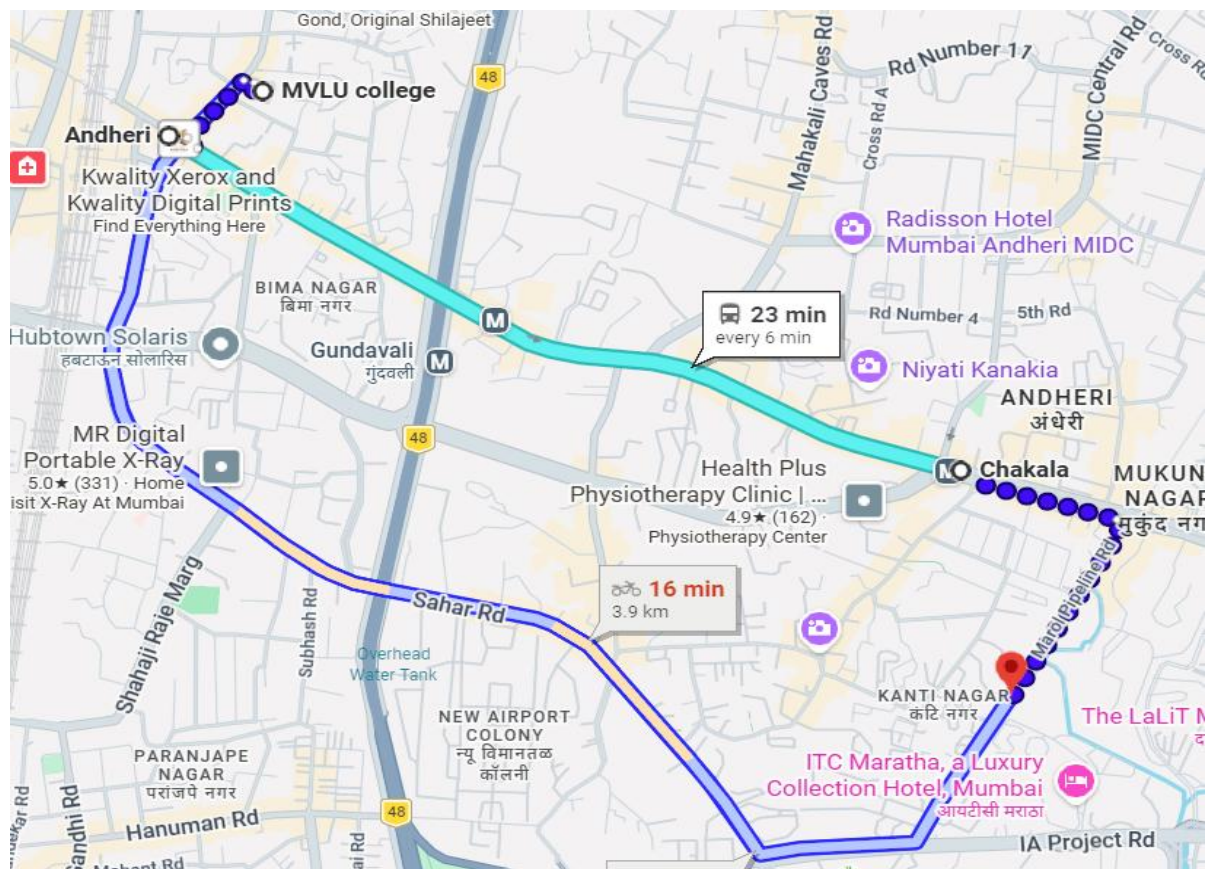
Estimated Time: 16 minutes

Suggested Route:

College -Xerox Shop A - Chakala – Mukund

Nagar C – Home

(Insert Google Maps screenshot here.)



A* Search Output

```
Path : College -> Xerox Shop -> Chakala -> Mukund Nagar -> Home  
Total Cost : 11  
Nodes Explored : 6  
Sagar Kandari T010
```

Source: College

Destination: Home

Path Found: College -> Xerox Shop -> Chakala -> Mukund Nagar -> Home

College - Junction A - Signal B - Flyover C -

Home Total Cost (Distance): 3.9 km

Nodes Explored: 5

Theory Questions - **(Find answer, justify them, if needed solve on paper and add images in your document. Don't need to write answers on paper. Directly add in documentation)**

1. Which algorithm would you use for the following applications? Explain

why.

- GPS Navigation

Algorithm: A*

Reason: It considers both the actual distance travelled and the estimated remaining distance, making it efficient for finding optimal routes.

- File System Search

Algorithm: DFS

Reason: DFS explores one directory completely before moving to another and requires less memory.

- Maze Solving

Algorithm: BFS

Reason: BFS guarantees the shortest path in an unweighted maze.

- Emergency Ambulance Routing

Algorithm: UCS or A*

Reason: They find the minimum-cost route considering road distances or travel time.

- Web Crawling

Algorithm: BFS

Reason: BFS visits webpages level by level and is commonly used by search engines.

2. The graph is the same, and the goal is the same (College to Home). The only thing that changes is the data structure:

- BFS - Queue
- DFS - Stack
- UCS/Greedy Best First/A* - Priority Queue

Why do you think changing just one thing causes all the algorithms to behave differently? Explain in your own words.

Ans: The data structure determines which node is expanded next.

- **BFS** uses a **Queue (FIFO)**, so nodes are explored level by level.
- **DFS** uses a **Stack (LIFO)**, so it follows one path deeply before backtracking.
- **UCS** uses a **Priority Queue** ordered by total path cost.
- **Greedy Best-First Search** uses a **Priority Queue** ordered by heuristic value.
- **A*** uses a **Priority Queue** ordered by $g(n) + h(n)$.

Because the node selection strategy changes, the path found, total cost, and number of explored nodes can also change.

3. What is the difference between:

Shortest Path	Least-Cost Path	Fastest Path
Fewest edges or minimum distance	Minimum total cost	Minimum travel time
Usually used in unweighted graphs	Used in weighted graphs	Depends on traffic and road conditions

4. True or False. Give reason and explanation for your answer:

- BFS always finds the least-cost path.

BFS always finds the least-cost path.

False

Reason: BFS finds the shortest path only in unweighted graphs. It ignores edge costs.

- DFS always finds the shortest path.

False

Reason: DFS follows one branch deeply and may miss a shorter path.

- UCS always finds the minimum-cost path.

True

Reason: UCS always expands the lowest-cost node first, guaranteeing the least-cost path when edge costs are non-negative.

- Greedy Search is always optimal.

False

Reason: It considers only the heuristic value and ignores the actual path cost, so it may choose a suboptimal path.

- A* Search uses both cost and heuristic values.

True

Reason: A* calculates $f(n) = g(n) + h(n)$, combining the actual path cost and the estimated remaining cost to efficiently find an optimal path.